

Abstract

In today's agricultural sector, automation plays a vital role in improving productivity and reducing manual effort. This project focuses on the design of a smart automated flower harvesting system that aims to increase efficiency, accuracy, and consistency in floriculture. The system uses modern technologies to identify mature flowers and harvest them with minimal damage, reducing dependence on manual labor.

A camera module is used to capture real-time images of flowers in the field. These images are processed using image processing and machine learning techniques to determine the ripening of flowers. Once a suitable flower is detected, the control unit sends commands to a robotic mechanism that performs precise and controlled picking actions. This ensures uniform harvesting while maintaining flower quality.

The automated system helps overcome common challenges such as labor shortages, human fatigue, and inconsistent harvesting results. By improving speed and accuracy, the system supports sustainable farming practices and enhances overall productivity. The use of adaptive learning techniques allows the system to improve its performance over time.

Overall, this project demonstrates how automation, artificial intelligence, and precision control can be effectively applied to agriculture. The proposed flower harvesting system offers a scalable and efficient solution and highlights the potential of smart farming technologies in modern agricultural applications.

Table of Contents

	Page No.
Acknowledgment	i
Abstract	ii
Table of Contents	iii
List of Figures	iv
List of Acronyms and Abbreviations	v
Chapter 1 Introduction	1
1.1 The Present Study	1
Chapter 2 Literature Review	3-9
2.1 General Introduction	3
2.2 Literature survey	3
2.3 Summary	9
Chapter 3 Problem Statement and Objective	10
3.1 Problem statement	10
3.2 Aim of the work	10
3.3 Objective	10
Chapter 4 Methodology	11-29
Chapter 5 Result	30-33
Chapter 6 Conclusion and scope of future work	34-36
6.1 Conclusion	34
6.2 Scope of future work	35
References	
Appendix	
Plagiarism Report	
Personal Profile	

List of Figures

Figure No.	Description	Page No.
Figure 4.1	Block Diagram of an Automated Flower Picker for Precision Harvesting	13
Figure 4.2	Flow Chart of an Automated Flower Picker for Precision Harvesting	14
Figure 4.3	Raspberry Pi 3 board	15
Figure 4.4	MG995 servo motor	16
Figure 4.5	SG90 servo motor	17
Figure 4.6	DC motor	18
Figure 4.7	L298N motor driver	19
Figure 4.8	USB webcam	20
Figure 4.9	Battery	21
Figure 4.10	Switch	21
Figure 5.1	Prototype of Automated Flower Picker for Precision Harvesting	31

List of Acronyms and Abbreviations

AI	Artificial Intelligence
ANN	Artificial Neural Network
CNN	Convolutional Neural Network
DC	Direct Current
DOF	Degree of Freedom
GPIO	General Purpose Input Output
GPS	Global Positioning System
GUI	Graphical User Interface
IMU	Inertial Measurement Unit
IoT	Internet of Things
IR	Infrared
LBP	Local Binary Pattern
LiDAR	Light Detection and Ranging
ML	Machine Learning
MJPEG	Motion Joint Photographic Experts Group
OS	Operating System
PWM	Pulse Width Modulation
RNN	Recurrent Neural Network
ROS	Robot Operating System
RTOS	Real-Time Operating System
USB	Universal Serial Bus
VNC	Virtual Network Computing
YOLO	You Only Look Once

INTRODUCTION

Flower farming is an important agricultural activity, but the process of harvesting flowers is still mostly done by hand. Manual harvesting takes a lot of time, requires continuous effort, and depends heavily on human availability. During peak seasons, labor shortages often create serious problems for farmers. Human fatigue and differences in skill levels can also lead to uneven quality, which affects both productivity and profit.

To solve these challenges, this project introduces an automated flower picker designed for precision harvesting. The system uses modern technologies such as robotics, embedded systems, and image processing to identify and pick flowers accurately. Instead of relying fully on human workers, the robot performs harvesting tasks in a controlled and consistent manner, reducing physical effort and saving time.

At the core of the system is a Raspberry Pi, which acts as the brain of the robot. A USB camera connected to it continuously captures live images of flowers in the field. These images help the system visually understand the surroundings and guide the harvesting process. The camera feed also allows real-time monitoring, which improves accuracy and safety during operation.

The robot is mounted on a mobile platform driven by DC motors. These motors allow smooth forward, backward, and turning movements so the robot can navigate between plants. A motor driver ensures that high power is safely handled without damaging the control unit. Speed control and safety limits help the robot move steadily even on uneven agricultural land.

A robotic arm fitted with servo motors performs the actual picking operation. Larger servo motors handle the main arm movements, while smaller servos manage fine actions such as gripping. The arm is designed to move smoothly and gently, ensuring flowers are picked without damaging petals or surrounding plants. This gentle handling helps maintain flower quality, which is often difficult in manual harvesting.

The system follows a clear working flow. First, the camera scans the field for flowers. Once a suitable flower is identified, the robotic arm positions itself carefully. The flower is then

cut and placed into a collection basket. After completing one cycle, the system automatically returns to scanning mode, allowing continuous operation without interruption.

Software plays a key role in coordinating all activities. Image processing using OpenCV enables live video streaming and visual guidance. A web-based interface and remote access tools allow the user to control, monitor, and debug the system wirelessly. This makes the robot easy to operate without the need for physical monitors or keyboards in the field.

The results of the project show that the automated flower picker works reliably and efficiently. The robot moves smoothly, the arm performs precise actions, and the system runs continuously without fatigue. Compared to manual harvesting, it provides better consistency, reduced labor dependency, and improved operational efficiency.

In conclusion, the automated flower picker demonstrates that smart farming solutions can effectively support modern agriculture. The system is cost-effective, scalable, and adaptable for future upgrades such as AI-based flower detection and autonomous navigation. With further improvements, this technology has strong potential to improve productivity and sustainability in floriculture.

LITERATURE REVIEW

2.1 General Introduction

The literature review plays an important role in the framework of the Automated Flower Picker of Precision Harvesting project since it ensures a strong definition of the existing level of technologies and studies in the sphere of smart agriculture. It emphasizes the progress in Artificial Intelligence (AI), Machine Learning (ML), and robotics playing the key role in the development of this project. The team can learn the best practices and prevent some pitfalls of implementing the flower harvesting by studying the similar case of automation. It assists in the legitimization of the applications of such components as Raspberry Pi, Arduino, and TensorFlow in reference to successful applications in the past. Another drawback of the literature review is that the past methods have some difficulties like damaging of delicate flowers or low efficiency of the image processing. This helps the project to overcome such gaps. Knowledge of past studies will make the proposed system innovative and based on proven methods. It also promotes sustainability objectives of the project as it implements effective resource utilization plans that had been established in previous researches. All in all, the literature review confirms that the approach of the project is valid and helps to make it have technical and practical significance in the modern agribusiness.

2.2 Literature Survey

2.2.1 “Conceptual design of a rose harvesting robot for greenhouses” Cahit GürelProf. Dr. Abdulkadir ErdenAtılım University, Mechatronics Engineering Dept.[1]

This work presents an automated robotic system designed to replace manual rose harvesting in greenhouse environments. The system aims to reduce labor dependency and improve harvesting efficiency. It is structured into three main subsystems for effective operation. A computer vision unit is used to detect the ripeness of roses accurately. A precision cutting mechanism ensures clean stem cutting without damaging the plant. A picking unit safely holds and transports harvested flowers. The design addresses challenges such as variations in stem length. It also considers the presence of different rose varieties in greenhouses. Quick post-harvest handling is emphasized to maintain flower freshness.

A top-down design methodology is followed for systematic development. Flowcharts are used to represent the operational sequence of the system. Petri nets help in modeling coordination and system behavior. These modeling techniques improve reliability and performance evaluation. Cost effectiveness is considered for practical greenhouse implementation. Overall, the system provides an efficient and reliable solution for automated rose harvesting.

2.2.2 The design and simulation of rose harvesting robot yerrapothu manikanta, student, seeram srinivasa rao prof & r. venkatesh prof, Department of Mechanical Engineering, Koneru Lakshmaiyya , Vaddeswaram , Vijayawada, Guntur, Andhra Pradesh, India.[2]

An automated robotic system is proposed to address labor shortages, high operational costs, and physical difficulties associated with manual rose harvesting. The system focuses on improving efficiency and reducing human effort in agricultural operations. Machine learning techniques are used to identify fully bloomed roses accurately. Gaussian Naive Bayes classification is applied for decision making during flower detection. Image processing algorithms implemented in MATLAB support accurate visual analysis. A camera-equipped manipulator is used to detect rose locations in real time. The robotic end-effector performs precise cutting of rose stems. Harvested roses are safely collected and offloaded by the system. Autonomous route planning enables the robot to move without human assistance. Control algorithms are implemented using KEIL programming tools. The mechanical structure of the robot is designed using CREO software. Structural strength and performance are evaluated using ANSYS simulations. The system is capable of continuous operation under different conditions. Automation reduces dependency on skilled labor. It improves harvesting speed and consistency. Operational costs are significantly reduced through automation. The robot minimizes human physical strain. The system supports modern agricultural practices. It demonstrates practical feasibility for real-world deployment. Overall, the approach enhances productivity in automated rose harvesting.

2.2.3 Saffron Harvesting using Robotic Arm Embedded on a Rover Ms. J. Priyadharshini student, Mr. T.ARUNN student, Mr. S. Vinoth kumar Prof, Mr. B. Rithick roshan Student, Department of Electronics And Communication Engineering, Sri Sairam Engineering College Tambaram, Chennai.[3]

An automated robotic approach is proposed to simplify the harvesting of saffron, which is traditionally a labor-intensive and time-consuming process. The system integrates a six degree-of-freedom robotic arm mounted on an autonomous rover. Embedded controllers such as Raspberry Pi and Arduino are used for system control and coordination. The robotic arm is designed to perform precise picking operations. A soft silicon gripper is used to avoid damage to delicate saffron flowers. CNN-based image processing is applied for identifying mature flowers. Real-time visual data is captured using onboard cameras. The vision system locates the exact position of the flowers. The robotic arm plucks flowers based on the processed visual information. Autonomous navigation of the rover is achieved using sensors. GPS and IMU sensors help in field movement and positioning. The system operates without continuous human supervision. A custom dataset is used to train the image processing model. Experimental results show an accuracy of about 95 percent. High precision and efficiency are observed during trials. Automation reduces manual labor significantly. The system improves harvesting consistency and reliability. Predictability of harvesting output is enhanced. The approach supports modernization of saffron farming. Overall, the system provides a feasible and efficient solution for automated saffron harvesting.

2.2.4 Real Time Farmer Assistive Flower Harvesting Agricultural Robot” Bhaskar S, Pradeep Kumar M, Avinash M N, Harshini S B Dept of ECE, SJGIT Chikkaballapura, India.[4]

An intelligent agricultural robot is developed to automate flower harvesting and assist farmers with multiple farm operations. The system focuses on reducing manual effort and improving harvesting efficiency. Image processing techniques are used to identify flowers accurately. Machine learning methods such as Local Binary Pattern and TensorFlow are applied for classification. The robot detects healthy flowers and avoids damaged or diseased ones. Embedded platforms including Raspberry Pi, Arduino, and NodeMCU are used for control and communication. Various sensors are integrated for environmental monitoring. The robot is capable of detecting the presence of pests in the field. Based on detection, it can spray insecticides automatically. Soil parameters such as moisture and condition are monitored. The collected data helps in improving crop management decisions. Internet of Things technology is used for real-time communication. Farmers receive updates and alerts on their smartphones. The system operates in real time with minimal human intervention. Automation improves accuracy and consistency in

harvesting. The solution reduces labor dependency and operational cost. AI-based decision making enhances reliability. The system supports sustainable agricultural practices. It improves productivity and resource utilization. Overall, the approach provides a cost-effective and modern solution for smart farming.

2.2.5 "Innovative robot for chamomile flower harvesting as a new approach based on visual selection", Gamal E. M. Nasr, Mohamed M. Ibrahim, Agricultural Engineering Department, Faculty of Agriculture, Cairo University, 12613, Egypt, Mohamed Y. Tayel, Dina S. Salama Water Relation and Field Irrigation Department, Agricultural and Biological Research Institute, National Research Centre, 12622, Egypt.[5]

An autonomous robotic system is developed to reduce labor cost and improve efficiency in chamomile flower harvesting. The system focuses on visual selection to accurately identify flowers suitable for harvesting. A mobile robotic platform is used for field operation. The picking mechanism follows a delta architecture to achieve fast and precise movements. Vision-based detection is implemented using deep learning techniques. The YOLOv4 model is applied for real-time flower detection. Image processing is performed efficiently using NVIDIA Jetson Nano. Arduino UNO is used for motion control and coordination. The robot is capable of identifying flowers in complex field environments. Real-time processing allows quick decision making during harvesting. The system achieves an average harvesting time of about three seconds per flower. A success rate of approximately 89.7 percent is reported during experiments. The robot is capable of harvesting nearly 1200 flowers per hour. Visual perception accuracy is measured at around 72.4 percent. Overall harvesting efficiency reaches nearly 80 percent. This performance is significantly higher than manual harvesting methods. Automation improves consistency and speed of operation. The system reduces dependency on skilled labor. It supports sustainable agricultural practices. Overall, the approach demonstrates strong potential for modern automated chamomile farming.

2.2.6 "Robotic Harvesting of Rosa Damascena Using Stereoscopic Machine Vision" Armin Kohan, Ali Mohammad Borghaee, Department of Agricultural Machinery Eng., College of Agriculture, Mehran Yazdi, Department of Communications and Electronics Eng, Saeid Minaei of Agricultural Machinery Eng, College of Agriculture, Tarbiat Modares Univ., Tehran, Iran. Mohammad Javad Sheykhdavudi, Department of Agricultural Machinery Eng., College of Agriculture, Chamran Univ., Ahvaz, Iran.[6]

An automated robotic system is developed for harvesting Rosa Damascena using stereoscopic machine vision. The system aims to reduce labor dependency and improve harvesting accuracy. Two cameras are used to capture stereo images of flowers in the field. Three-dimensional information is extracted from the stereo vision setup. This depth information helps in accurately locating flowers in complex environments. Image processing techniques are applied to detect flowers among leaves and branches. A four-degree-of-freedom robotic arm is used for harvesting operations. The robotic arm is equipped with a custom-designed end-effector. The end-effector performs precise picking of flowers without causing damage. Camera placement and distance play a critical role in detection accuracy. Experimental results show optimal performance with a 100 mm distance between cameras. The system achieves a harvesting success rate of about 82.22 percent. Accurate depth estimation improves picking precision. Automation allows consistent harvesting without human fatigue. The system operates effectively in outdoor farming conditions. It reduces errors caused by manual harvesting. The approach improves efficiency and flower quality. Stereoscopic vision enhances reliability in complex scenes. The system demonstrates practical feasibility for rose harvesting automation. Overall, the solution supports modern intelligent agricultural practices.

2.2.7 “Flower Bot: A Deep Learning aided Robotic Process to detect and pluck flowers”, R Raja Subramanian, Department of Computer Science and Engineering, Nali Harsha Vardhan, Department of Computer Science and Engineering Kalasalingam Academy of Research and Education Virudhnagar, Tamilnadu, India.[7]

An autonomous robotic system is developed to detect and pluck flowers using deep learning techniques. The system aims to overcome difficulties such as thorns, physical strain, and poor visibility conditions. A Raspberry Pi is used as the main processing unit. A night-vision camera enables flower detection even in low-light environments. Image processing is performed using OpenCV for real-time analysis. A convolutional neural network is trained for flower classification. The CNN model is trained using the Oxford 102 flower dataset. The trained model achieves a detection accuracy of about 92 percent. A PIC microcontroller is used to control the robotic arm movements. The robotic arm is designed to perform precise plucking operations. Servo motors are used to control the cutters and arm joints. The system uses an excavator chain motion for effective flower searching. Flowers are detected and plucked within approximately six seconds.

Automation reduces manual effort and increases operational safety. The system minimizes damage caused by thorns. Low-cost hardware components are used to improve affordability. The system supports night-time agricultural operations. Detection accuracy is improved through deep learning. The approach enhances harvesting efficiency and consistency. Overall, the system provides a reliable solution for automated flower harvesting.

2.2.8 Image Processing Based Rose Harvesting System using Raspberry Pi, Charu Bhartiya MTech, Embedded System VIT University Vellore, Tamil Nadu, India, Prof. Ashish P. Assistant Professor, School of Electronics Engineering VIT University Vellore, Tamil Nadu, India.[8]

An automated rose harvesting system is developed using image processing techniques on a Raspberry Pi platform. The system focuses on identifying roses through live image capture. A webcam is used to acquire real-time images of flowers. OpenCV is implemented in C++ for image processing operations. A cascade classifier based on Local Binary Pattern is used for rose detection. The system is trained using a limited dataset of rose images. When a rose is detected, control signals are generated automatically. An LED indicator is activated to indicate successful detection. A servo motor is triggered to perform the cutting operation. The harvesting process is simple and automated. The system achieves an accuracy of approximately 70 percent. Hardware limitations affect detection performance. Lighting conditions influence the accuracy of image detection. The approach demonstrates low computational complexity. The system is suitable for low-cost implementations. It highlights the feasibility of Raspberry Pi-based vision systems. Suggestions include improving classifiers for better accuracy. Increasing training data can enhance detection reliability. Upgrading hardware can improve processing speed. Overall, the system provides a basic and cost-effective solution for automated rose harvesting.

2.2.9 Design and Implementation of Autonomous Flower Harvester using Image Processing, A VinothKumar, V Kannarasu, S Padmapriya, N Partheeban, S Arun.[9]

An autonomous flower harvesting system is developed using image processing techniques for efficient harvesting of small flowers. The system is designed as a low-cost robotic platform suitable for agricultural applications. Image processing is used to detect the presence and location of flowers accurately. The watershed algorithm is applied to segment flower regions from the background. Histogram distance matching is used to compare

flower similarity. Harvesting action is triggered when similarity reaches 70 to 80 percent. The system calculates the centroid of the detected flower region. The cutting point is determined based on the centroid position. A robotic arm is directed toward the cutting point for harvesting. The robotic arm is controlled using an Arduino microcontroller. A motorized cutter is used to perform the cutting operation. Morphological image processing operations improve detection accuracy. Conversion of images to grayscale simplifies processing. The system performs real-time harvesting without collisions. Automation reduces dependency on manual labor. The approach improves harvesting precision and consistency. The system is adaptable to different flower types. It supports real-time field operation. The design emphasizes simplicity and cost effectiveness. Overall, the system demonstrates reliable performance for autonomous flower harvesting.

Summary

The literature survey highlights extensive research carried out in the field of automated flower harvesting using robotics, image processing, and artificial intelligence. Several studies focus on replacing manual harvesting with robotic systems to reduce labor dependency and improve efficiency. Vision-based techniques using machine learning, CNNs, and image processing algorithms such as LBP, watershed, and histogram matching are widely used for flower detection and ripeness identification. Researchers have employed platforms like Raspberry Pi, Arduino, PIC controllers, and NVIDIA Jetson for real-time processing and control. Robotic arms with soft grippers, cutters, and precision end-effectors are commonly adopted to avoid damage to delicate flowers. Autonomous rovers equipped with GPS, IMU, and sensor-based navigation enable field movement and scalability. Advanced systems using stereoscopic vision and YOLO-based models have demonstrated higher accuracy and faster harvesting rates. Some solutions also integrate IoT for farmer assistance, monitoring, and pest management. Despite high accuracy, challenges such as cost, hardware limitations, lighting variations, and flower diversity still exist. Many studies emphasize the need for cost-effective, adaptable, and reliable designs for real-world deployment. Drawing inspiration from these works, the proposed project aims to combine AI-based vision, robotic precision, and embedded control to achieve accurate and gentle flower harvesting. The system focuses on improving consistency, efficiency, and flower quality. Adaptive learning and real-time monitoring further enhance performance. Overall, the reviewed literature validates the feasibility and importance of intelligent automation in floriculture.

Problem Statement and Objective

3.1 Problem statement:

Manual flower harvesting is labor-intensive, inconsistent, and costly, with a growing shortage of skilled labor. An automated flower picker ensures precise, efficient, and consistent harvesting, reducing costs and improving quality.

3.2 Aim of the work:

“Automated Flower Picker for Precision Harvesting,” is a smart robotic system that can pick flowers on its own, quickly and accurately, without needing much help from people. The idea is to use technologies like robotics, sensors, and image processing to find and pick only the fully bloomed flowers, without harming the rest of the plant. By making the picking process more precise, this system can help save time, cut down on labor costs, and make sure the flowers that are picked are consistently high in quality.

3.3 Objective:

To develop an automated system that accurately identifies and harvests mature flowers with minimal human intervention.

Methodology

The methodology of the automated flower picker project defines a systematic approach for designing, integrating, and operating the system to achieve precision harvesting. It begins with hardware selection and system integration, where the Raspberry Pi 3 is chosen as the central controller due to its processing capability and GPIO support. A USB webcam is integrated for visual sensing, servo motors are selected for precise robotic arm movement, and DC motors with a motor driver are used for system mobility.

The next stage of the methodology involves vision-based flower detection. Real-time images captured by the camera are processed using image-processing algorithms to identify flowers based on features such as color, shape, and size. Image pre-processing steps like noise reduction and resizing are applied to improve detection accuracy. This stage is critical for ensuring that only the target flowers are selected for harvesting.

Once a flower is detected, the methodology focuses on decision-making and motion control. The Raspberry Pi processes the detection results and generates control signals for the robotic arm. PWM signals are used to control servo motors, enabling smooth and accurate positioning of the arm for cutting and collection. Simultaneously, DC motors controlled through the motor driver allow the system to reposition itself when required.

The final stage of the methodology covers actuation, harvesting, and system reset. The robotic arm performs controlled cutting and places the flower into the collection basket. After completing one harvesting cycle, the system returns to scanning mode, allowing continuous operation. Power management and safety considerations are included throughout the methodology to ensure stable and reliable performance.

The importance of this methodology lies in its structured and sequential nature. It ensures proper coordination between hardware and software components, improves accuracy in flower detection, minimizes damage during harvesting, and enhances system reliability. A well-defined methodology also simplifies testing, debugging, and future expansion, making the project practical and scalable for real-world agricultural automation.

4.1 Block diagram

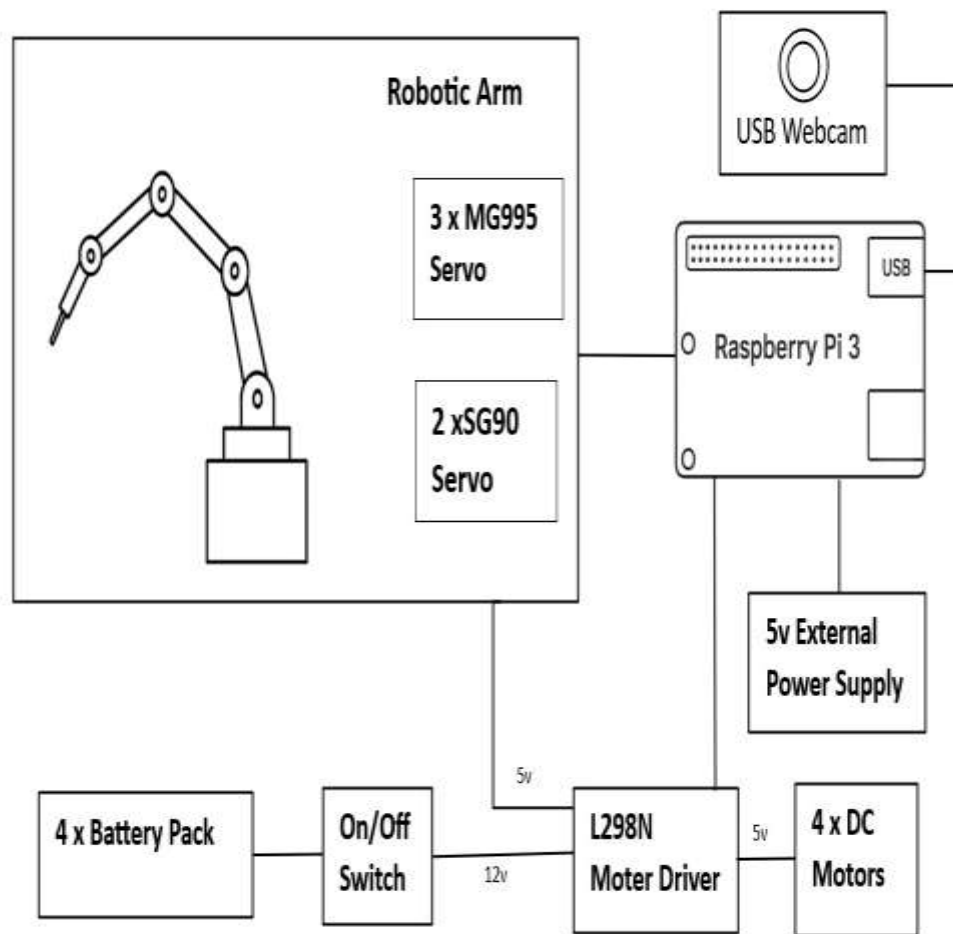


Figure 4.1: Block diagram of Automated flower picker for precision harvesting

4.1.1 Hardware Components

1. Raspberry Pi 3 board
2. MG995 servo motors
3. SG90 servo motors
4. DC motors
5. L298N motor driver
6. USB webcam
7. Battery
8. Switch

4.2 Flow chart

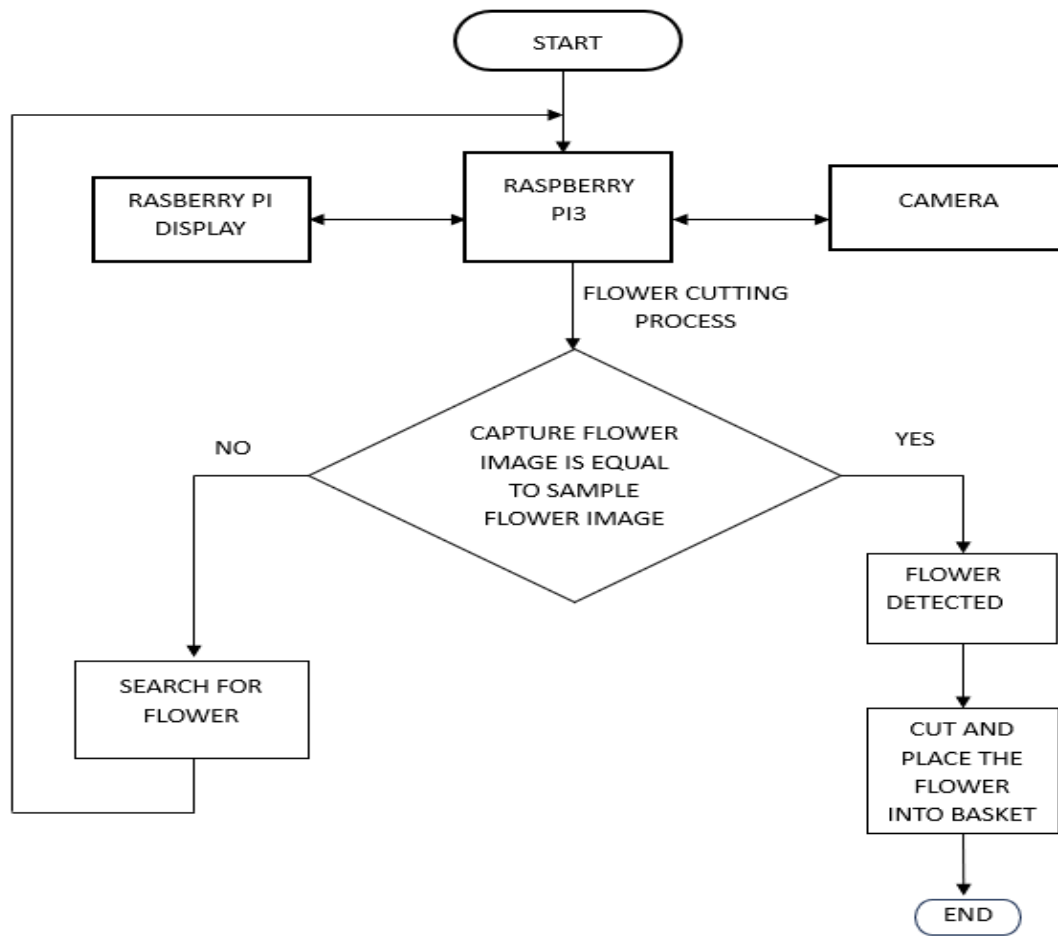


Figure 4.2: Flow chart of Automated flower picker for precision harvesting

Steps:

1. Image Acquisition:

The USB camera acts as the image sensing hardware and continuously captures real-time images of flowers present in the field. These images are transferred to the Raspberry Pi through the USB interface. The Raspberry Pi receives the image frames and prepares them for further software-based processing.

2. Image Display and Monitoring:

The Raspberry Pi processes the incoming image stream and forwards it to the connected display unit. The display shows the live camera feed along with system-generated operational messages. This hardware–software interaction enables real-time monitoring of the flower detection and harvesting process.

3. Image Pre-processing:

The software running on the Raspberry Pi performs image pre-processing operations such as resizing, filtering, and noise reduction. These operations enhance image quality and remove unwanted disturbances, thereby improving the accuracy of subsequent image analysis.

4. Feature Extraction and Image Comparison:

Image-processing algorithms extract important visual features such as color, shape, and size from the captured flower image. These extracted features are compared with a pre-stored sample flower image available in the Raspberry Pi memory. This software-based comparison helps in accurately identifying the required flower.

5. Decision Making:

Based on the image comparison result, the software executes a decision-making logic. If the captured image does not match the reference image, the camera continues scanning and sending new images. If a match is confirmed, the system proceeds to the actuation stage.

6. Control Signal Generation:

After successful flower detection, the Raspberry Pi generates digital control signals through its GPIO pins. These signals act as an interface between the software logic and the hardware cutting mechanism.

7. Flower Cutting Operation:

The cutting mechanism receives control signals from the Raspberry Pi and performs a precise cutting operation. Software-controlled timing ensures smooth movement and prevents damage to the flower and surrounding plants during harvesting.

8. Flower Collection:

Once the flower is cut, it is automatically guided into the collection basket. This operation is synchronized by the control software to ensure completion of one harvesting cycle.

9. Completion and System Reset:

After completing the harvesting process, the software confirms successful operation. The system then returns to scanning mode and waits for the next flower detection cycle.

4.3 Components Description



Figure 4.3: Raspberry Pi 3 board

The Raspberry Pi 3 is a potential controller of the automated flower picker system of the precision harvesting; it is a small powerful single-board computer. It serves as the brain of the robot as it interprets the information and makes smart decisions. It operates on a 5-volt stable power supply in a mobile-type adapter keeping the board safe and operated continuously. Raspberry Pi 3 operates the operating system and manages the general workflow of the machine in this system. It is connected to USB camera which takes real time pictures of flowers in the field. It is done by the means of image processing and AI to recognize the flowers and determine their ripeness. The board implements TensorFlow-based models to determine whether or not a flower should be picked. According to this decision, there is the production of control signals to the robotic arm. Raspberry Pi passes on this control data in the form of the GPIO pin to the servo motors or motor drivers. It is not in direct power of heavy motors since it cannot sustain high currents. They are instead safely operated by external motor drivers. Other board controllers such as Arduino or node MCU are also made available to the board as necessary. PIR and IR sensors gather sensor data that is read to identify movement and positioning. Raspberry Pi takes control of time, coordination, and synchronization of each component. It aids in real time monitoring and data listing to be analyzed. This is because of its small size so there is easy mounting on the harvesting robot. Generally, the Raspberry Pi 3 is critical towards effective, precise, and intelligent harvesting of flowers.



Figure 4.4:MG995 Servo Motor

In the automated flower picker system, MG995 servo motor is a significant actuator that will be applied to pick the flowers precisely. It is primarily utilized in the manipulation of the bigger joints of the robotic arm like the lifting and bending types of movements. This servo motor is characterized by high torque, which means it is able to support the arm weight, and other flower plucking mechanism. The MG995 is connected with three wires: power, ground and signal. A 5-volt external adaptor is used to provide the power wire with a steady and reliable power source. External power takes up excess load and safe guard the control board. The controller ground completes the electrical circuit with the ground wire. Control pulses are sent to the signal wire by the main controller, i.e. the Raspberry Pi or Arduino. The angle and the position of the motor shaft are determined by these pulses. The MG995 does not rotate continuously, as is the case with DC motors. It can only turn to certain angles hence control of arm movements is very precise. This precision is necessary to pick flowers carefully without any damages. The servo reacts rapidly to the change of command allowing to the uniquely fluid and regulated movement. It assists in locating the gripper precisely close to the target flower. The gears in the MG995 are made of strong metal that enhances durability when performing a repetitive action. The motor also enhances constant and stable operations in field agriculture. It enables the joint movements of more than one arm. The MG995 servo motor in general is a strong and very selective muscle of the robotic arm making harvesting of the flowers, care, accurate and efficient.



Figure 4.5 SG90 Servo Motors

SG90 servo motor is a lightweight and small motor that is used in the automated flower picker system and is applied to precisely harvest their products. It is employed mostly when fine and precise movement is required like in the control of the wrist, gripper or end-effector of the robotic arm. SG90 is also applicable to the job where smooth movement is required instead of heavy lifting. It can work with the 5-volt supply of power supplied safely via an external adapter line. Three wires are used to connect the motor, namely power, ground, and signal. The neutral of the controller should be connected to the ground of the controller so that the circuit is complete. Pulse-width commands are sent to the signal wire to the controller that contains a Raspberry Pi or Arduino. These indicators determine the angle to which the servo will rotate. The SG90 has an ability to move to certain angles/rotate, typically 0 to 180 degrees. This is a rotary motion of the gripper, which aids to open and close it softly. Flows are very sensitive and therefore smooth and slow motions are most valued. The SG90 offers stable and vibration free movement and minimizes the chances of damage to the flowers. It does not have to bear heavy weight but it is effective in light work. Its small size can easily be mounted in the immediate surrounding of the gripper system. It uses less power to consume energy and thus the system is energy efficient. The servo is responsive to minor commands to movement. This enhances precision on flower picking. On the whole, the SG90 servo will resemble the soft and precise robotic finger. It is one of the factors that lead to proper handling and harvesting of flowers.



Figure 4.6: DC Motor

The four DC motors are heavy motors which are applied in the automated flower picker system so as to give movement and mechanical rotation. They are primarily operated to move the wheels of the robot to enable it move forward, back and turn in the field. The motors are engineered to endure constant rotary and greater loads than the servo motors. Large wires are used to connect each DC motor to the output of the motor driver board to enable the higher current to be taken safely. The motors consume a 12-volt supply of power which is supplied through a battery pack. Connection is not made with the controller directly since DC motors consume high current. The motor driver in turn serves as an interface between the controller and the motors. The motor driver generates the direction of motor rotation according to the controller. Pulse signals to the driver board are used to control the speed. As opposed to the servo motors, the DC motors do not shift to some fixed angles. They are free moving as long as some power is supplied to them. This contributes to their use in the tasks of movement of wheels and mechanical driving. The motors are started and stopped as per the commanding the controller via the driver. They give the required pull to propel the robot over rough farmlands. It is fitted with four motors making it more balanced and stable. This assists the robot to go through flower plants evenly. The motors are constructed to work outdoors. They transform electrical energy into powerful mechanical force. Altogether, the DC motors are the prime drivers of the automated flower picker and are the keys to stable movement and effective field performance to achieve accuracy.

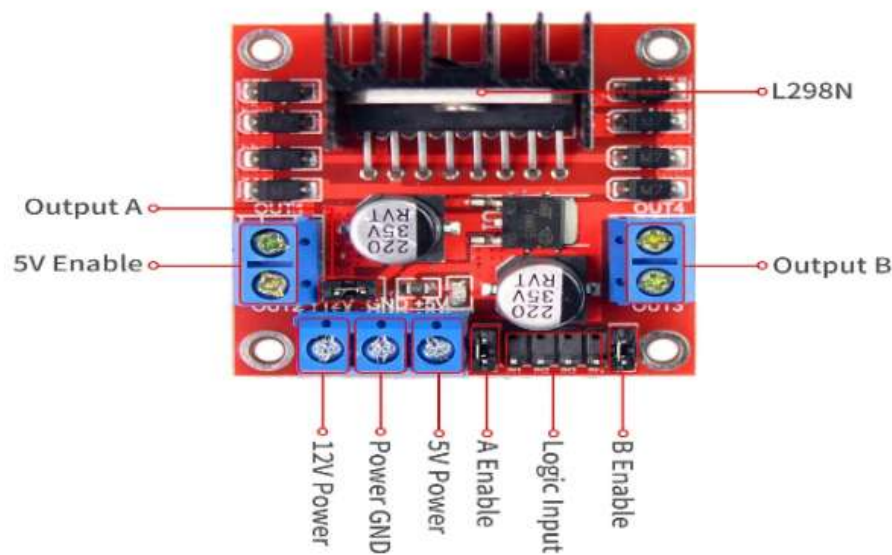


Figure 4.7: L298N Motor Driver

The L298N motor driver is a part of the automated flower picker system applied in the automated harvesting system to select the flower. It operates as a medium between the unit of controls and high-power DC motors. The driver assists in the safe input of motors which demand an increase in current and voltage. The L298N is directly connected to the M2 motors by the use of a 12-volt battery supply. This helps in avoiding the direct flow of high current into the Raspberry Pi. Its driver also takes a supply of 5 volt to its internal logic and control operations. The output of the Raspberry Pi sends control signals to the input pins of the L298N. Depending on such signals, the driver will dictate how the motors will rotate. It enables the rotating of the motors both forward and reverse. Pulse-width modulation signals are used to control the speed of the motors. L298N is able to operate 2 DC motors simultaneously. This causes it to be appropriate in moving the various wheels of the robot. The driver also takes care of the motor start and stop functioning. It shields the controller against the fluctuation of voltage and overloading. This safeguard is significant in the case of dependable system performance. The driver facilitates the stable movement of the robot at farm fields. It assists the robot to drive in-between plants. Harvesting is enhanced through proper motor control. The L298N is simple to combine with embedded systems. All in all, it makes the motor bearable, controlled, and efficient in the automated flower picker.

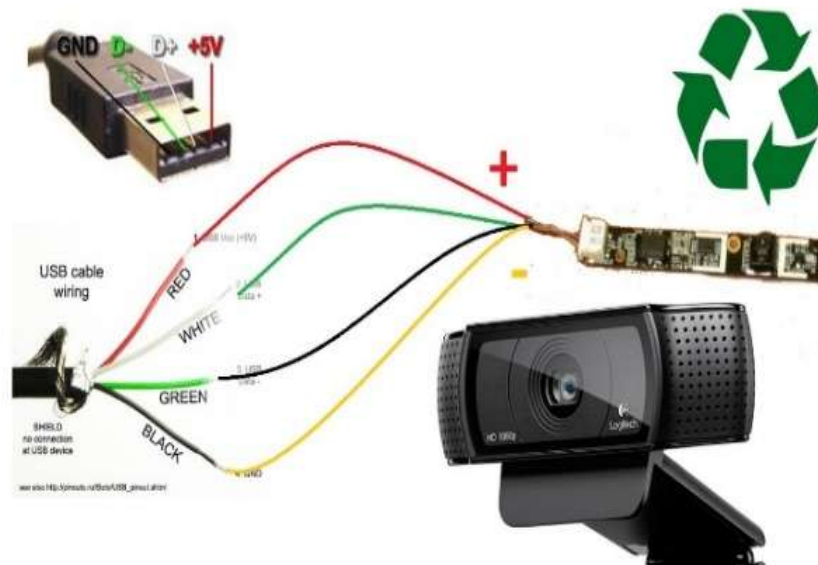


Figure 4.8: USB Webcam

Another significant sensing device of the automated flower picker is the USB webcamp, which serves as an ideal tool of picking the flowers accurately. It is the vision component of the robot, and this enables it to see what is in its environment. The camera is plugged to the Raspberry Pi via a USB cable. Through the same USB interface, it is powered and connected to the computer with data. The webcam records the real-time video and still photos of flowers in the field. This is sent to the Raspberry PI to process this visual data constantly. The camera is not controlled by motors and other mechanical pieces. It can purely serve a visual informative purpose. Raspberry Pi uses this data of the image to process the image and analyze it through AI technology. On the basis of this data, the system recognizes flowers and verifies their location and maturity. TensorFlow models scan the images and determine whether a flower needs harvesting. The camera assists in finding precise whereabouts of the flower. This enables the robotic arm to target the target precisely. The harvesting process can be followed in real-time, with the help of the webcam. It assists in preventing hurdles and missteps. Accuracy in detection is enhanced by accurate lighting and camera angle. The camera provides accurate and meticulous picking of flowers. It enhances the human reliance on sight. Its small size enables it to be mounted on the robot. The overall effect is that the USB webcam serves as the eyes of the robot and therefore allows the intelligent and precise harvesting of accuracy.



Figure 4.9: Battery

The 12 V battery pack supplies only the required DC power to operate the DC motors through the motor driver. It is connected through switch to safely control the power flow in the system. The battery is not connected directly to the Raspberry Pi because the voltage level is not suitable for the controller. Instead, the motor driver receives the 12 V DC supply and distributes it safely to the motors. This power is sufficient to run the wheel motors smoothly and steadily. The battery acts as the main power source for robot movement during operation.



Figure 4.10: Switch

The switch is an electromechanical component used to control the power supply of the system. It is connected in series with the power line to enable or interrupt current flow. When the switch is in the ON state, the circuit is completed and power is delivered to the system. When the switch is in the OFF state, the circuit is open and power flow is stopped. This prevents unintended operation during setup or maintenance. The switch provides a simple and reliable method for power control.

4.4 Software Description

4.4.1 Algorithm for Rose detection

1. **System Initialization:** Initialize the Raspberry Pi system and load required libraries for image processing, motor control, servo control, web communication, and GPIO handling.
2. **GPIO and Hardware Configuration:** Configure GPIO pins for DC motors and servo motors. Initialize PWM signals for motor speed control and servo angle positioning.
3. **Simulation Handling (Safety Mode):** If GPIO hardware is not detected, switch the system to simulation mode to allow software testing without physical hardware.
4. **Camera Initialization:** Initialize the USB camera and set the resolution parameters. Allow sufficient delay for stable camera startup.
5. **Web Server Initialization:** Start the Flask web server to provide a user interface for monitoring and control of the robot.
6. **Socket Communication Setup:** Establish real-time communication using Socket IO to receive control commands and send system status updates.
7. **Live Video Capture:** Continuously capture frames from the camera. Encode the frames and stream live video to the web interface.
8. **Mode Selection:** Set the default operating mode to manual. Allow switching between manual and automatic modes through the web interface.
9. **Manual Motion Control:** In manual mode, receive joystick input values for movement. Apply speed limiting and calculate left and right motor speeds using differential drive logic.
10. **DC Motor Control:** Generate PWM signals based on calculated speed values. Control motor direction using GPIO logic signals.
11. **Robotic Arm Servo Control:** Receive servo angle commands from the user interface. Convert angle values to PWM duty cycles and position the arm joints accordingly.
12. **Gripper Operation:** Control the gripper servo to open or close based on user command for picking or releasing flowers.
13. **Safety Speed Limitation:** Restrict motor speed to safe limits to prevent sudden movements and ensure stable operation during harvesting.
14. **Continuous Monitoring:** Monitor system operations and print debug information for motor speeds and servo positions.

15. **Error Handling:** Detect and handle runtime errors during motor or servo operations to prevent system crashes.
16. **Graceful Shutdown Detection:** Monitor for interrupt signals such as keyboard interrupts or stop commands.
17. **Motor Stop Procedure:** Stop all DC motors immediately when shutdown is initiated to ensure safety.
18. **Servo Shutdown:** Stop all servo PWM signals to prevent unnecessary power draw and mechanical stress.
19. **GPIO Cleanup:** Reset all GPIO pins to safe states and release hardware resources properly.
20. **Camera Release:** Release the camera resource and close all OpenCV windows.

4.4.2 Algorithm for Robotic Arm Control Using Servo Motors

1. **Initialization:** Start the program and initialize the GPIO library in BCM mode to enable pin numbering for Raspberry Pi.
2. **Servo Pin Configuration:** Define GPIO pins for base, shoulder, elbow, wrist, and gripper servo motors used in the robotic arm.
3. **PWM Setup:** Configure each servo pin as an output and initialize PWM signals at 50 Hz, which is suitable for servo motor control.
4. **Servo Activation:** Start PWM for all servo motors with an initial duty cycle of zero to keep the arm in a stable position.
5. **Initial Angle Assignment:** Set default angular positions for shoulder, elbow, wrist, and gripper to ensure a safe starting posture of the arm.
6. **Base Rotation Control:** Rotate the base servo left or right by applying predefined angles for horizontal movement of the arm.
7. **Shoulder Joint Control:** Increase or decrease the shoulder joint angle incrementally while ensuring the angle remains within safe limits.
8. **Elbow Joint Control:** Control the elbow joint movement by adjusting its angle to raise or lower the arm section.

9. **Wrist Joint Control:** Adjust the wrist angle to fine-tune the orientation of the end-effector during flower picking.
10. **Gripper Control:** Open or close the gripper by setting appropriate servo angles to gently hold or release flowers.
11. **Angle-to-PWM Conversion:** Convert desired servo angles into corresponding PWM duty cycles using a predefined formula.
12. **Servo Movement Execution:** Apply the calculated duty cycle to the servo motor and maintain it for a fixed duration to allow physical movement.
13. **Stabilization Delay:** Introduce a time delay after each movement to ensure smooth and accurate positioning.
14. **PWM Reset:** Reset the duty cycle to zero after movement to prevent servo jitter and overheating.
15. **Incremental Motion Control:** Use small angle increments for precise and controlled movement of arm joints.
16. **Safety Limits Enforcement:** Restrict servo angles within defined limits to avoid mechanical damage.
17. **Continuous Operation:** Repeat movement commands based on user input or control logic during harvesting operations.
18. **Task Completion:** Complete the picking or placing operation once the gripper action is executed.
19. **Cleanup Procedure:** Stop all PWM signals after operation is completed.
20. **GPIO Resource Release:** Clean up all GPIO resources to safely terminate the program.

4.5 Image Processing Techniques

The automated flower harvesting system uses image processing techniques primarily for real-time visual monitoring and guidance. Image acquisition is performed using a USB camera interfaced with a Raspberry Pi. The camera continuously captures live video frames

of the flower field, providing visual input to the system. OpenCV is used as the core image processing library for handling camera operations and frame manipulation.

Each captured frame is read using OpenCV's video capture interface and stored as a digital image matrix. The resolution of the captured images is fixed at 640×480 pixels to balance clarity and processing speed. The acquired frames are encoded into JPEG format using OpenCV's image encoding functions. This compression reduces data size and enables smooth real-time transmission.

The encoded image frames are streamed over a web interface using the Motion JPEG (MJPEG) streaming technique implemented through a Flask web server. MJPEG streaming sends a sequence of JPEG images, allowing continuous real-time video display in a browser without complex decoding requirements. This provides the operator with live visual feedback during robot movement and flower picking.

The image processing in the current implementation is used for visual assistance rather than automatic flower detection. The live video stream helps the user manually align the robotic arm with the target flower, improving precision during harvesting. The camera feed also aids in navigation and obstacle awareness.

All image processing tasks are software-based and executed on the Raspberry Pi using OpenCV. The system is designed in a modular way so that advanced image processing algorithms such as color segmentation, feature extraction, or deep learning-based flower detection can be integrated in future work. Thus, the image processing module forms a foundation for intelligent vision-based harvesting.

4.6 Use of Video Capture

The `VideoCapture.read()` function from the OpenCV library is used to acquire real-time video frames from the camera. In this project, a USB webcam is interfaced with the Raspberry Pi, and `cv2.VideoCapture(0)` is initialized to access the camera device. The `read()` method continuously captures individual frames from the live video stream.

The function returns two values: a boolean flag and an image frame. The boolean flag indicates whether the frame has been successfully captured, while the frame contains the actual image data in the form of a matrix. If the flag is false, it indicates a failure in frame capture.

Each successfully captured frame is stored temporarily in memory and used for further processing or display. The captured frames are then encoded and streamed to the web interface for real-time monitoring. This allows continuous visual feedback to the user during robot operation.

The `VideoCapture.read()` method plays a crucial role in enabling live image acquisition, forming the foundation for visual monitoring and future vision-based flower detection in the automated harvesting system.

4.7 Use of RealVNC Viewer

RealVNC Viewer is used in this project as the primary interface to access and control the Raspberry Pi installed on the automated flower harvesting robot. Since the Raspberry Pi is mounted on the mobile robotic platform, it is not practical to connect a dedicated monitor, keyboard, or mouse during operation. RealVNC Viewer enables remote desktop access to the Raspberry Pi over a local Wi-Fi network, allowing full graphical control from a laptop or PC.

Through RealVNC Viewer, the user can view the Raspberry Pi desktop in real time and interact with it exactly as if working directly on the board. All project-related Python programs, including motor control, robotic arm control, camera handling, and web server execution, are run through this remote interface. The Flask and SocketIO-based control application is launched via the Raspberry Pi terminal accessed using RealVNC Viewer. Once the application is running, it continuously listens for control inputs and executes motor and servo commands accordingly.

The robotic model movement is enabled because the control code runs directly on the Raspberry Pi hardware. When commands are issued (either through the web interface or manual triggers), the Raspberry Pi processes them and sends electrical control signals to the GPIO pins. These GPIO signals drive the motor driver for DC motors and PWM signals for servo motors controlling the robotic arm. Since the code execution and GPIO access happen on the Raspberry Pi itself, RealVNC Viewer acts as the bridge that allows the user to start, stop, and monitor these processes remotely.

The live camera feed captured using OpenCV is also monitored through applications launched via RealVNC Viewer. This allows real-time observation of the field environment

and assists in accurate positioning of the robot and arm. Debug messages, motor speed values, and servo angle updates are visible on the terminal through the VNC session, which helps in testing and troubleshooting.

In summary, RealVNC Viewer does not directly control the motors but provides a remote operating environment through which all control software is executed. It plays a critical role in enabling wireless operation, real-time monitoring, and safe control of the robotic system, making it an essential tool for the implementation and testing of the automated flower harvesting project.

4.8 Equation used for System Design

1. Servo Angle: PWM Duty Cycle Equation

$$\text{Duty Cycle (\%)} = 2.5 + \text{Theta}/18$$

Where

Theta = servo angle in degrees(0-180)

Purpose: This equation converts the desired servo angle into a PWM duty cycle so the servo rotates to the required position.

2. Motor Speed Limiting

$$x = \text{clip}(x, -1, 1) * 0.5$$

$$y = \text{clip}(y, -1, 1) * 0.5$$

Purpose:

- Limits joystick input to safe range
- Reduces speed to 50%
- Prevents sudden or unsafe motion

3. Differential Driver Motor Speed Equation

$$VL = (y+x)*100$$

$$VR = (y-x)*100$$

Where

- x = turning command
- y = forward/backward command
- VL = left motor speed(%)
- VR = right motor speed(%)

Purpose

- Move forward/backward
- Turn left/right
- Perform smooth curved motion

4. Angular Saturation Equation

$$\theta = \min (\theta_{\max}, \theta + \Delta\theta)$$

$$\theta = \max (\theta_{\min}, \theta - \Delta\theta)$$

Purpose

- Prevents servo rotation beyond mechanical limits
- Protects robotics arm joints

5. Incremental Angle Control Equation

$$\Theta_{\text{new}} = \theta_{\text{current}} \pm \Delta\theta$$

Purpose

- Allows smooths step-by-step movement
- Avoids sudden jerks in robotic arm motion

6. Wrist Rotation Constraints Equation

$$0 \leq \theta_{\text{wrist}} \leq 90^\circ$$

Purpose

- Maintain natural wrist orientation
- Avoids cable twisting and servo strain

Summary

The servo angle to PWM duty cycle equation, $\text{Duty Cycle} = 2.5 + \frac{\theta}{18}$, is used to control the position of the servo motors in the robotic arm. In this equation, the servo angle θ ranges from 0° to 180° , and the corresponding PWM duty cycle is generated to rotate the servo to the required position. This ensures accurate and smooth angular movement of the arm joints and gripper.

The motor speed limiting equations, $x = \text{clip}(x, -1, 1) \times 0.5$ and $y = \text{clip}(y, -1, 1) \times 0.5$, are used to restrict joystick inputs within a safe range. By scaling the input to 50%, the robot's speed is reduced, which prevents sudden acceleration and ensures stable and safe movement during operation.

The differential drive motor speed equations, $V_L = (y + x) \times 100$ and $V_R = (y - x) \times 100$, control the speed of the left and right motors. Here, x represents the turning command and y represents forward or backward motion. These equations allow the robot to move straight, turn left or right, and perform smooth curved movements effectively.

The angular saturation equations, $\theta = \min(\theta_{\max}, \theta + \Delta\theta)$ and $\theta = \max(\theta_{\min}, \theta - \Delta\theta)$, are applied to limit servo angles within their mechanical boundaries. This prevents the robotic arm joints from rotating beyond safe limits and protects the servos from mechanical damage.

The incremental angle control equation, $\theta_{\text{new}} = \theta_{\text{current}} \pm \Delta\theta$, enables step-by-step movement of the robotic arm joints. By updating angles gradually, the arm moves smoothly and avoids sudden jerks, improving precision and stability during operation.

Finally, the wrist rotation constraint equation, $0^\circ \leq \theta_{\text{wrist}} \leq 90^\circ$, limits the wrist movement to a safe range. This maintains proper wrist orientation, avoids excessive twisting, and ensures reliable handling of objects during the harvesting process.

RESULT

5.1 Result Analysis

The Automated Flower Picker for Precision Harvesting project successfully achieved its primary objective of developing a functional robotic system capable of assisting flower harvesting operations. The integration of embedded hardware components such as the Raspberry Pi 3, DC motors, servo motors, motor driver, robotic arm, and USB camera resulted in a stable and operational prototype. The system demonstrated coordinated performance between sensing, control, actuation, and monitoring modules, validating the feasibility of automation in floriculture applications.

From an operational perspective, the mobile robotic platform exhibited smooth and controlled movement using differential drive logic. The DC motors driven through the L298N motor driver responded accurately to control signals, enabling forward, backward, and turning motions required for field navigation. Speed limiting and PWM-based control ensured stable motion and reduced mechanical stress, making the system suitable for continuous operation in agricultural environments.

The robotic arm mechanism produced precise and reliable movements during testing. Servo-controlled joints for the base, shoulder, elbow, wrist, and gripper allowed accurate positioning and gentle handling of flowers. The gripper mechanism performed effectively in minimizing physical damage, which is a major drawback of manual harvesting. Incremental angle control and stabilization delays ensured smooth motion, contributing to consistent flower quality and reduced plant damage.

The image processing module based on OpenCV successfully provided real-time visual monitoring through live video streaming. The USB webcam captured continuous frames, which were encoded and streamed using the MJPEG technique via a Flask web interface. Although automatic flower detection was not implemented in the current version, the visual feedback enabled accurate manual alignment and navigation, forming a strong foundation for future AI-based detection and classification.

System usability and control were significantly enhanced through software integration. Flask and SocketIO enabled real-time web-based interaction, while RealVNC Viewer allowed remote access to the Raspberry Pi for executing, monitoring, and debugging

control programs. This wireless control approach eliminated the need for physical peripherals and enabled flexible system operation, especially during field testing and experimentation.

The project demonstrated improved efficiency, consistency, and reliability compared to manual flower harvesting methods. The prototype successfully reduced dependency on human labor, maintained uniform harvesting quality, and operated continuously without fatigue. The results confirm that the proposed system is technically viable, scalable, and adaptable for future enhancements, contributing meaningfully to smart farming and precision agriculture solutions.



Figure 5.1: Prototype of Automated Flower Picker for Precision Harvesting

5.1.1 Evaluation of Automated flower picker

The evaluation of image fusion techniques highlights the strengths and weaknesses of different approaches used to combine information from multiple images into a single, more informative output. Traditional spatial-domain techniques such as simple averaging, addition, and maximum selection are widely used due to their simplicity and low

computational cost. However, these methods often struggle with preserving fine details and tend to produce fused images with reduced contrast or unwanted noise.

Their performance is acceptable only when the input images are very similar in structure and lighting conditions. Transform-domain methods provide a significant improvement by decomposing images into frequency components before fusion. Techniques like wavelet transform, Discrete Cosine Transform (DCT), Principal Component Analysis (PCA), and Laplacian pyramid fusion are more effective at retaining important features such as edges, texture, and high-frequency information. Wavelet-based fusion, in particular, is known for its ability to preserve both spatial and spectral details, making it suitable for applications like medical imaging, remote sensing, and surveillance. However, these methods can be computationally intensive, and their performance depends heavily on the choice of decomposition level and fusion rules.

5.2 Summary

The Automated Flower Picker for Precision Harvesting is a robotic system developed to improve efficiency, accuracy, and consistency in floriculture by reducing dependence on manual labor. The system is designed to selectively identify, pick, and collect flowers with minimal damage to the plant. By automating the harvesting process, it addresses key challenges such as labor shortages, inconsistent harvesting quality, and time-intensive manual operations.

The system integrates computer vision techniques using OpenCV for real-time image acquisition and visual analysis. Captured video frames are processed to support flower identification, ripeness assessment, and position estimation. The vision module provides spatial information that assists in accurate alignment of the robotic arm. This visual feedback plays a critical role in improving picking precision and reducing errors during harvesting operations.

A multi-degree-of-freedom robotic arm driven by servo motors performs controlled plucking actions. The servomotors are controlled using PWM signals generated by the embedded controller, enabling smooth and precise joint movements. The gripper mechanism is designed to handle flowers gently, minimizing physical damage during picking. Motion control logic ensures accurate positioning of the arm based on visual input and predefined movement sequences.

The overall system follows a structured harvesting workflow consisting of scanning, detecting, positioning, picking, and storing. Path planning and controlled motion reduce unnecessary movement and avoid contact with unripe or damaged flowers. The modular hardware–software architecture allows future integration of advanced sensors and intelligent algorithms, making the system scalable and suitable for precision agriculture and smart farming applications.

Conclusion and Scope of the Future Work

6.1 Conclusion

The Automated Flower Picker for Precision Harvesting project successfully demonstrates the integration of embedded hardware and intelligent software into a functional robotic prototype. The system combines a Raspberry Pi controller, DC motors, servo motors, a robotic arm, and a camera to perform controlled movement and flower harvesting operations. This integration validates the practical feasibility of automation in floriculture.

The mobile platform uses differential drive control to achieve smooth and stable navigation in the field. DC motors controlled through a motor driver enable precise forward, backward, and turning motions. Speed regulation and safety limits ensure controlled movement, reducing mechanical stress and improving reliability during operation.

The robotic arm mechanism plays a vital role in the harvesting process. Servo motors are used to control the base, shoulder, elbow, wrist, and gripper joints, allowing accurate positioning and gentle handling of flowers. Incremental angle control and PWM-based actuation ensure smooth movements and minimize the risk of damaging delicate flowers.

Image processing using OpenCV provides real-time video acquisition and visual feedback. Live camera streaming supports manual alignment of the robotic arm and assists in navigation. Although automatic flower detection is not implemented in the current version, the image processing module forms a strong foundation for future integration of advanced AI-based detection techniques.

Software integration is achieved through Flask and SocketIO, which provide a web-based interface for real-time control and monitoring. RealVNC Viewer enables remote access to the Raspberry Pi, allowing wireless execution, supervision, and debugging of the control programs. This approach eliminates the need for physical peripherals and enhances system flexibility.

Overall, the project confirms that automated flower harvesting is technically achievable using cost-effective components. The modular design allows scalability and future enhancements such as machine learning-based flower detection and autonomous

navigation. The developed prototype contributes toward smart farming solutions by improving efficiency, consistency, and sustainability in agricultural operations.

6.2 Scope of the Future Work

Future enhancements of the system will focus on improving hardware accuracy and software efficiency. Advanced motor control techniques such as closed-loop feedback using encoders can be implemented to improve positional accuracy and movement stability. Fine-tuning PWM frequency, torque control, and servo calibration will enhance the precision of robotic arm operations and reduce mechanical wear.

The image processing module can be upgraded from visual monitoring to full autonomous flower detection and classification. Integration of deep learning models such as Convolutional Neural Networks (CNNs) or YOLO-based object detection can enable real-time identification of flower type, ripeness, and location. Training the model with a larger, diverse dataset will improve detection accuracy under varying lighting and environmental conditions.

Navigation and mobility can be enhanced by incorporating autonomous path planning algorithms. Sensors such as ultrasonic sensors, LiDAR, GPS, and IMU can be used for obstacle detection, localization, and field mapping. This will allow the robot to navigate large agricultural fields independently without manual control.

The user interface can be further developed into a complete human–machine interaction system. Enhancements such as live system status indicators, error notifications, and performance dashboards can improve operational transparency. Mobile application integration can also enable remote monitoring and control through smartphones.

Energy optimization is another key area for future work. Implementing intelligent power management, sleep modes, and energy-efficient components can significantly extend battery life. Integration of solar panels can further support sustainable and long-duration field operation.

Finally, system scalability and reliability can be improved through modular hardware and software design. Multi-robot coordination can be explored to increase harvesting efficiency. Cloud-based data storage and IoT integration can enable long-term data analysis

and predictive maintenance. Detailed technical documentation and standardized interfaces will support easier deployment, maintenance, and future expansion of the system.

Reference

- [1] “Conceptual design of a rose harvesting robot for greenhouses” Cahit Gürel Prof. Dr. Abdulkadir Erden Atılım University, Mechatronics Engineering Dept.
- [2] The design and simulation of rose harvesting robot yerrapothu manikanta, student, seeram srinivasa rao prof & r. venkatesh prof, Department of Mechanical Engineering, Koneru Lakshmaiyya , Vaddeswaram , Vijayawada, Guntur, Andhra Pradesh, India.
- [3] Saffron Harvesting using Robotic Arm Embedded on a Rover Ms. J. Priyadharshini student, Mr. T. ARUNN student, Mr. S. Vinoth kumar Prof, Mr. B. Rithick roshan Student, Department of Electronics And Communication Engineering, Sri Sairam Engineering College Tambaram, Chennai.
- [4] Real Time Farmer Assistive Flower Harvesting Agricultural Robot” Bhaskar S, Pradeep Kumar M, Avinash M N, Harshini S B Dept of ECE, SJ CIT Chikkaballapura, India.
- [5] "Innovative robot for chamomile flower harvesting as a new approach based on visual selection”, Gamal E. M. Nasr, Mohamed M. Ibrahim, Agricultural Engineering Department, Faculty of Agriculture, Cairo University, 12613, Egypt, Mohamed Y. Tayel, Dina S. Salama Water Relation and Field Irrigation Department, Agricultural and Biological Research Institute, National Research Centre, 12622, Egypt.
- [6] “Robotic Harvesting of Rosa Damascena Using Stereoscopic Machine Vision” Armin Kohan, Ali Mohammad Borghae, Department of Agricultural Machinery Eng., College of Agriculture, Mehran Yazdi, Department of Communications and Electronics Eng, Saeid Minaei of Agricultural Machinery Eng, College of Agriculture, Tarbiat Modares Univ., Tehran, Iran. Mohammad Javad Sheykhdavudi, Department of Agricultural Machinery Eng., College of Agriculture, Chamran Univ., Ahvaz, Iran.
- [7] “Flower Bot: A Deep Learning aided Robotic Process to detect and pluck flowers”, R Raja Subramanian, Department of Computer Science and Engineering, Nali Harsha Vardhan, Department of Computer Science and Engineering Kalasalingam Academy of Research and Education Virudhnagar, Tamilnadu, India.
- [8] Image Processing Based Rose Harvesting System using Raspberry Pi, Charu Bhartiya MTech, Embedded System VIT University Vellore, Tamil Nadu, India, Prof. Ashish P.

Assistant Professor, School of Electronics Engineering VIT University Vellore, Tamil Nadu, India.

- [9] Design and Implementation of Autonomous Flower Harvester using Image Processing, A VinothKumar, V Kannarasu, S Padmapriya, N Partheeban, S Arun.
- [10] Zhou, H., Wang, S., & Li, D. (2024). Construction and Optimization of a Collaborative Harvesting System for Multiple Robotic Arms and an End-Picker in a Trellised Pear Orchard Environment. *Agronomy*, 14(1), 80. <https://www.mdpi.com/2073-4395/14/1/80>
- [11] Yu, K., et al. (2020). Advancing Specialty Crop Management: A Review of Recent Developments in Robotics, Remote Sensing, and Machine Learning Systems. *International Food and Agribusiness Management Review*. <https://brill.com/view/journals/ifam/aop/article-10.22434-ifamr1104/article-10.22434-ifamr1104.xml>
- [12] Singh, P., Sharma, H., & Kuntal, P. (2021). Agricultural Robots for Harvesting and Planting. *Turkish Journal of Computer and Mathematics Education (TURCOMAT)*. <https://turcomat.org/index.php/turkbilmat/article/view/14410>
- [13] Kang, H., Zhou, H., & Chen, C. (2020). Visual Perception and Modeling for Autonomous Apple Harvesting. *IEEE Access*, 8, 62151–62163.
- [14] Mu, L., et al. (2020). Design and Simulation of an Integrated End-Effector for Picking Kiwifruit by Robot. *Information Processing in Agriculture*, 7, 58–71.
- [15] Yang, C. H., et al. (2020). Integrated Detection of Citrus Fruits and Branches Using a Convolutional Neural Network. *Computers and Electronics in Agriculture*, 174, 105469.
- [16] Bac, C. W., Van Henten, E., Hemming, J., & Edan, Y. (2014). Harvesting Robots for High- Value Crops: State-of-the-Art Review and Challenges Ahead. *Journal of Field Robotics*, 31, 888–911. <https://doi.org/10.1002/rob.21525>
- [17] Slamet, R. A., et al. (2019). An Automated Fruit Harvesting Robot by Using Deep Learning. *ROBOMECH Journal*, 6, 13. <https://doi.org/10.1186/s40648-019-0141-2>
- [18] Zhou, H., Wang, X., Au, W., et al. (2022). Intelligent Robots for Fruit Harvesting: Recent Developments and Future Challenges. *Precision Agriculture*, 23, 1856–1907. <https://doi.org/10.1007/s11119-022-09913-3>

- [19] Zhang, J., Kang, N., Qu, Q., et al. (2024). Automatic Fruit Picking Technology: A Comprehensive Review of Research Advances. *Artificial Intelligence Review*, 57, 54. <https://doi.org/10.1007/s10462-023-10674-2>

APPENDIX

1. Code for Rose Detection

```
import os
import cv2
import time
import signal
import sys
import threading
import numpy as np
from flask import Flask, render_template, Response, jsonify
from flask_socketio import SocketIO, emit
from dotenv import load_dotenv

# Try to import RPi.GPIO, use mock if not available
try:
    import RPi.GPIO as GPIO
    GPIO.setmode(GPIO.BCM)

    # Motor pins
    MOTOR_PINS = {
        'left': {'in1': 5, 'in2': 6, 'en': 20},
        'right': {'in1': 12, 'in2': 13, 'en': 21}
    }

    # Servo pins
    SERVO_PINS = {
        'base': 16,
        'shoulder': 17,
        'elbow': 18,
        'wrist': 19,
        'gripper': 23
    }

    # Setup motor pins
    for motor in MOTOR_PINS.values():
        GPIO.setup([motor['in1'], motor['in2'], motor['en']], GPIO.OUT)
        motor['pwm'] = GPIO.PWM(motor['en'], 1000)
        motor['pwm'].start(0)

    # Setup servo pins
    servos = {}
    for name, pin in SERVO_PINS.items():
        GPIO.setup(pin, GPIO.OUT)
```

```

servos[name] = GPIO.PWM(pin, 50)
servos[name].start(0)

def set_servo_angle(servo_name, angle):
    if servo_name in servos:
        duty = 2.5 + (angle / 18)
        servos[servo_name].ChangeDutyCycle(duty)

def set_motor_speed(motor_side, speed):
    motor = MOTOR_PINS[motor_side]
    speed = max(-100, min(100, speed))

    if speed > 0:
        GPIO.output(motor['in1'], GPIO.HIGH)
        GPIO.output(motor['in2'], GPIO.LOW)
    else:
        GPIO.output(motor['in1'], GPIO.LOW)
        GPIO.output(motor['in2'], GPIO.HIGH)

    motor['pwm'].ChangeDutyCycle(abs(speed))

def stop_motors():
    for motor in MOTOR_PINS.values():
        motor['pwm'].ChangeDutyCycle(0)

def cleanup():
    stop_motors()
    for motor in MOTOR_PINS.values():
        motor['pwm'].stop()
    for servo in servos.values():
        servo.stop()
    GPIO.cleanup()

print("RPi.GPIO initialized successfully")

except (ImportError, RuntimeError) as e:
    print(f"RPi.GPIO not available: {e}")
    print("Running in simulation mode")

def set_servo_angle(servo_name, angle):
    print(f"[SIM] {servo_name}: {angle}°")

def set_motor_speed(motor_side, speed):

```

```

    print(f"[SIM] {motor_side} motor: {speed}%")

def stop_motors():
    print("[SIM] Stopping motors")

def cleanup():
    print("[SIM] Cleanup")

# Initialize Flask app
app = Flask(__name__)
app.config['SECRET_KEY'] = os.getenv('SECRET_KEY', 'rose_harvester_secret')

# Initialize SocketIO with fallback
try:
    import eventlet
    eventlet.monkey_patch()
    socketio = SocketIO(app,
                        async_mode='eventlet',
                        cors_allowed_origins="*",
                        logger=True,
                        engineio_logger=True)
    print("Using eventlet")
except ImportError:
    socketio = SocketIO(app,
                        async_mode='threading',
                        cors_allowed_origins="*",
                        logger=True,
                        engineio_logger=True)
    print("Using threading")

# Global variables
camera = None
camera_lock = threading.Lock()
current_mode = 'manual'

# Initialize camera
def init_camera():
    global camera
    camera = cv2.VideoCapture(0)
    camera.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
    camera.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
    time.sleep(2)

```

```

# Video streaming generator
def gen_frames():
    global camera
    while True:
        with camera_lock:
            success, frame = camera.read()
            if not success:
                break
            ret, buffer = cv2.imencode('.jpg', frame)
            frame = buffer.tobytes()
            yield (b'--frame\r\n'
                   b'Content-Type: image/jpeg\r\n\r\n' + frame + b'\r\n')

# Routes
@app.route('/')
def index():
    return render_template('index.html')

@app.route('/video_feed')
def video_feed():
    return Response(gen_frames(),
                    mimetype='multipart/x-mixed-replace; boundary=frame')

# WebSocket events
@socketio.on('connect')
def handle_connect():
    print('Client connected')
    emit('mode_update', {'mode': current_mode})

@socketio.on('set_mode')
def handle_set_mode(data):
    global current_mode
    if data['mode'] in ['manual', 'auto']:
        current_mode = data['mode']
        emit('mode_update', {'mode': current_mode}, broadcast=True)
        print(f"Mode: {current_mode}")

# Update the handle_manual_control function
@socketio.on('manual_control')
def handle_manual_control(data):
    if current_mode == 'manual':
        try:
            # Apply 50% speed reduction and limit to safe range

```

```

x = max(-1, min(1, data.get('x', 0))) * 0.5 # 50% speed
y = max(-1, min(1, data.get('y', 0))) * 0.5 # 50% speed

# Calculate motor speeds with differential drive
left_speed = (y + x) * 100 # Convert to percentage
right_speed = (y - x) * 100 # Convert to percentage

# Apply the speeds
set_motor_speed('left', left_speed)
set_motor_speed('right', right_speed)

# Debug output
print(f'Motors - Left: {left_speed:.1f}%, Right: {right_speed:.1f}%')

except Exception as e:
    print(f'Error in manual control: {e}')

# Update the servo control function
@socketio.on('servo_control')
def handle_servo_control(data):
    if current_mode == 'manual':
        try:
            servo = data.get('servo')
            if 'angle' in data:
                angle = int(data['angle'])
                print(f'Setting {servo} to {angle} degrees')
                set_servo_angle(servo, angle)
            elif 'action' in data and servo == 'gripper':
                angle = 0 if data['action'] == 'open' else 90
                print(f'Gripper {data['action']} to {angle} degrees')
                set_servo_angle('gripper', angle)
        except Exception as e:
            print(f'Error in servo control: {e}')

def signal_handler(sig, frame):
    print("\nShutting down gracefully...")
    cleanup() # Call your existing cleanup function
    sys.exit(0)

# Register the signal handler
signal.signal(signal.SIGINT, signal_handler)

if __name__ == '__main__':

```



```

try:
    init_camera()
    # Run the SocketIO app
    socketio.run(app,
        host='0.0.0.0',
        port=5000,
        debug=True,
        use_reloader=False,
        allow_unsafe_werkzeug=True)
except KeyboardInterrupt:
    print("Shutting down...")
except Exception as e:
    print(f'Error: {e}')
finally:
    if 'camera' in globals() and camera:
        camera.release()
    if 'cleanup' in globals():
        cleanup()
    cv2.destroyAllWindows()

```

2. Code for Robotic Arm Control Using Servo Motors

```

# ===== arm_control.py =====
import RPi.GPIO as GPIO
import time

class ArmControl:
    def __init__(self):
        GPIO.setmode(GPIO.BCM)

        # Servo pins
        self.base_pin = 4
        self.shoulder_pin = 17
        self.elbow_pin = 27
        self.wrist_pin = 22
        self.gripper_pin = 23

        # Setup pins for PWM
        self.servos = {}
        for pin in [self.base_pin, self.shoulder_pin, self.elbow_pin, self.wrist_pin,
self.gripper_pin]:
            GPIO.setup(pin, GPIO.OUT)
            self.servos[pin] = GPIO.PWM(pin, 50) # 50 Hz

```

```

        self.servos[pin].start(0)

# Current angles
self.shoulder_angle = 130
self.elbow_angle = 120
self.wrist_angle = 90
self.gripper_angle = 0

# ----- Base Controls -----
def move_base_right(self, duration=0.1):
    self._move_servo(self.base_pin, 70, duration)

def move_base_left(self, duration=0.1):
    self._move_servo(self.base_pin, 110, duration)

# ----- Shoulder Controls -----
def shoulder_left(self, increment=5):
    self.shoulder_angle = min(180, self.shoulder_angle + increment)
    self._move_servo(self.shoulder_pin, self.shoulder_angle, 0.3)

def shoulder_right(self, decrement=5):
    self.shoulder_angle = max(0, self.shoulder_angle - decrement)
    self._move_servo(self.shoulder_pin, self.shoulder_angle, 0.3)

# ----- Elbow Controls -----
def elbow_down(self, increment=5):
    self.elbow_angle = min(180, self.elbow_angle + increment)
    self._move_servo(self.elbow_pin, self.elbow_angle, 0.3)

def elbow_up(self, decrement=5):
    self.elbow_angle = max(0, self.elbow_angle - decrement)
    self._move_servo(self.elbow_pin, self.elbow_angle, 0.3)

# ----- Wrist Controls -----
def wrist_up(self, increment=5):
    self.wrist_angle = min(90, self.wrist_angle + increment)
    self._move_servo(self.wrist_pin, self.wrist_angle, 0.3)

def wrist_down(self, decrement=5):
    self.wrist_angle = max(0, self.wrist_angle - decrement)
    self._move_servo(self.wrist_pin, self.wrist_angle, 0.3)

# ----- Gripper Controls -----

```

```

def gripper_open(self):
    print("gripper open")
    self.gripper_angle = 90
    self._move_servo(self.gripper_pin, self.gripper_angle, 0.3)

def gripper_close(self):
    print("gripper close")
    self.gripper_angle = 10
    self._move_servo(self.gripper_pin, self.gripper_angle, 0.3)

# ----- Internal Method -----
def _move_servo(self, pin, angle, duration):
    duty = 2 + (angle / 18) # Convert angle to duty cycle
    self.servos[pin].ChangeDutyCycle(duty)
    time.sleep(duration)
    self.servos[pin].ChangeDutyCycle(0)

def cleanup(self):
    for pwm in self.servos.values():
        pwm.stop()
    GPIO.cleanup()

```